

[What's new?](#) | [Help](#) | [Directory](#) | [Sign in](#)

mostpixelsever

Multi-Screen Library for Processing

 Search projects[Project Home](#)[Downloads](#)[Wiki](#)[Issues](#)[Source](#)

Search

Current pages

for

 Search

HowThisWorks

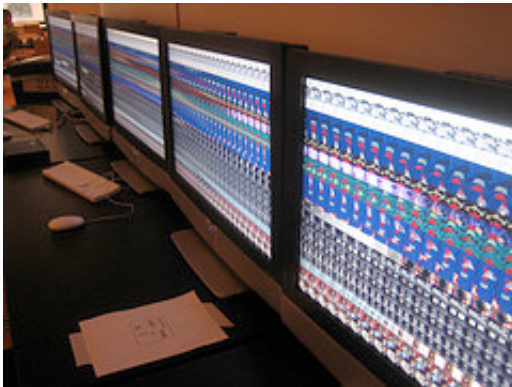
A description of the overall system, how it works, its pros and cons.

Updated Sep 23, 2008 by

[daniel.shiffman](#)Labels: [Featured](#), [Phase-Design](#)

What is this for?

The Most Pixels Ever library is a software solution for taking a single-screen real-time graphics application (developed in Java) and spanning it across multiple screens (each connected to a separate computer). Its features include “time syncing” and “spatial syncing.” Time syncing ensures that each frame is rendered simultaneously on each display and is achieved by network communication. Spatial syncing ensures that each client computer renders the appropriate portion of the larger display on its display.



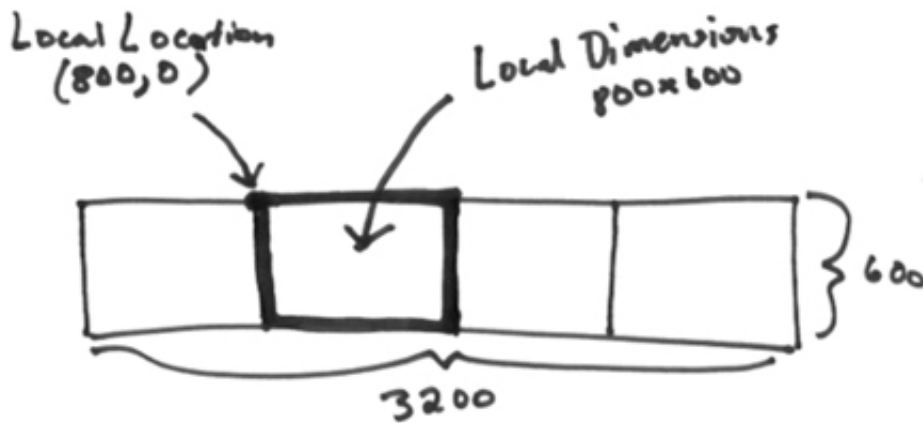
When using MPE, you only have to develop a single client application. The same client application can run on each computer. Each application, however, will load a initialization file that specifies its location, width, and height within the larger, master display.

Spatial Syncing: Master Dimensions and Local Dimensions

The driving force behind the MPE system are “master dimensions” and “local dimensions.” Master dimensions are defined as the total width and height (in pixels) of the space encompassed by all of the displays. Local dimensions, on the other hand, are the width and height of an individual display within the master dimensions.

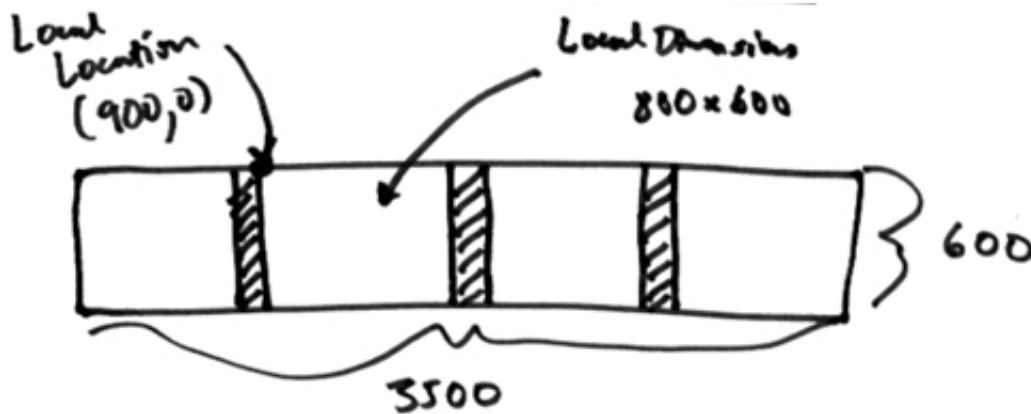
For example, consider a basic set-up of four LCD monitors (each set to 800 x 600) placed side by side in a horizontal line. The master dimensions of all four screens combined is therefore 3200 x 600. While the local dimensions for any given display remain 800 x 600, the local location differs from display to display.

Local locations: (0,0) (800,0) (1600,0) (2400,0).

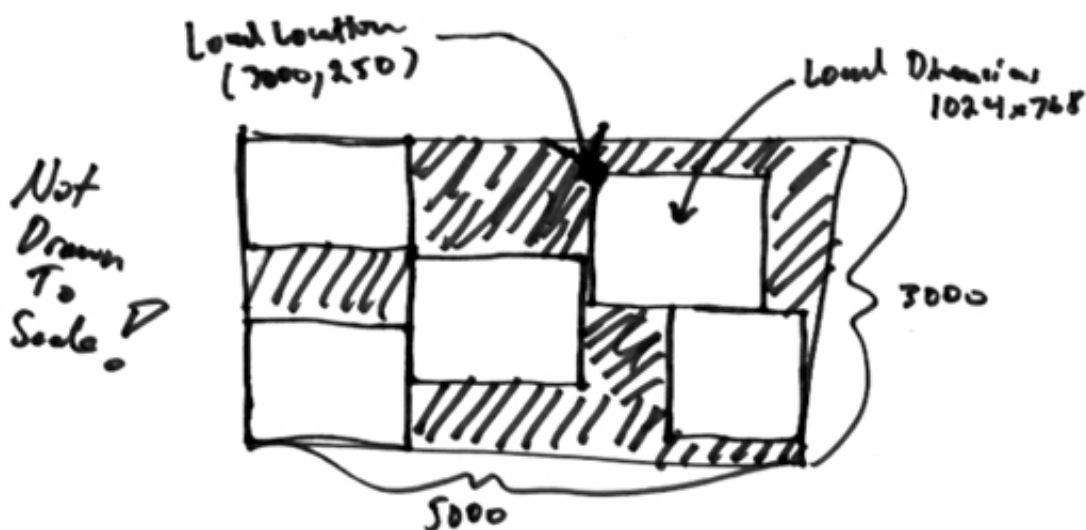


The Most Pixels Library, however, is a great deal more flexible than the above scenario. For example, what if you want to consider the bezels between each screen? After all, if you want objects to animate seamlessly from one display to the next, they will have to pass along the invisible space between the monitors. Armed with a ruler, you reasonably estimate the width of the screen's bezels in pixels (relative to the width of the actual display area itself). Let's assume you measure 100 pixels in between each screen. We now have:

Master Dimensions: $800 \times 4 + 100 \times 3 = 3500 \times 600$ pixels Local Dimensions: 800×600 Local Locations: $(0,0)$ $(900,0)$ $(1800,0)$ $(2700,0)$



Lining up screens horizontally is by no means required with MPE. The individual displays can be placed arbitrarily throughout the master space (and can be different sizes, dimensions, etc.)



Time Syncing

MPE works because computational media is inherently deterministic. If each client computer runs the same application and computes each frame at precisely the same time, the graphics will always stay synchronized. (If you use random numbers in your application, each client will need to seed the random generator with an identical value.)

In order for MPE to work, you must run the MPE server application (one of the client computers can also act as the server or you can use an additional machine). Based on a frame rate that you specify, the server broadcasts a message ("Render a new frame!") every N milliseconds to each client. The next frame will not be triggered until all clients have reported back that they have completed the current frame. MPE will only run as fast as the slowest computer connected to the system.

Admittedly this is inefficient. With MPE, although each machine only displays a portion of the larger image, it must compute everything for all displays. While this isn't ideal for performance, with MPE we are emphasizing simplicity above optimization. And for many multi-screen applications, the computation is easy enough for single machines to handle. (More advanced applications that require further optimization can be developed with MPE using the messaging system described below or by hacking the source.)

MPE offers a means of broadcasting messages between clients as well. For example, if you want to visualize information from an RSS feed or track a physical object with computer vision, you have a few options. The server can receive messages via a listening port and broadcast those messages to all clients. In addition, you can ask a single client to perform the data retrieval or analysis and broadcast the result to all clients. MPE only works if all clients perform all computations with the same exact data during the same exact frame. The server, therefore, is in charge of handing out all messages, locking them to specific frames.

For more details, check out the other tutorials as well as the MPE source.

Comment by [nanodust](#), Oct 08, 2008

why not just linux & use Xinerama ?

i mean - champ for re-writing all this... but it's been done in X11 for years & works super well.

Comment by [mangtronix](#), Oct 10, 2008

Here's a good reason:

<http://en.wikipedia.org/wiki/Xinerama> In most implementations, OpenGL (3D) direct-rendering only works on one of the screens. Windows that should show 3D graphics on other screens tend to just appear black. This is most commonly seen with 3D screen savers, which show on one of the screens and black on the others. (The Solaris SPARC OpenGL implementation allows direct rendering to all screens in Xinerama mode.)

Also with a lot of screens one machine probably can't render them all!

► [Sign in](#) to add a comment

©2009 Google - [Code Home](#) - [Terms of Service](#) - [Privacy Policy](#) - [Site Directory](#)

Hosted by 